



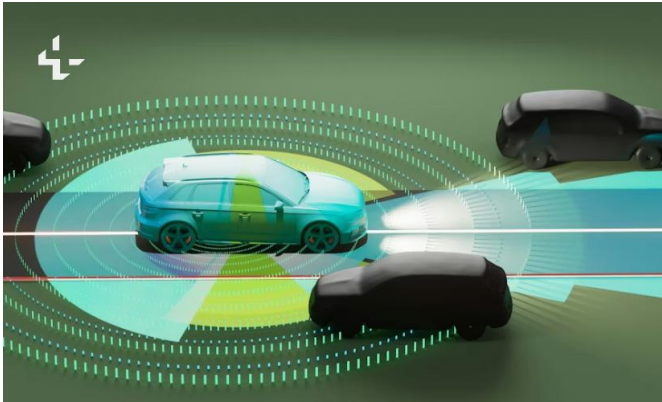
Sobel Edge Detection Accelerator on FPGA (PYNQ-Z2)

Lixuan Xu, Hao Ding, Yuhan Jiang

lx2349@nyu.edu, hd2891@nyu.edu, yj3494@nyu.edu

Background

- Edge detection is the first step in countless vision systems
- Identifies boundaries, shapes, and structures in images



Lane Detection



Object Recognition

Project Overview

A **fully pipelined** Sobel edge detection accelerator synthesized with Vitis HLS and deployed on PYNQ_Z2, processing grayscale images via AXI-Stream at **one pixel per clock cycle** using BRAM line buffers and a register-partitioned 3×3 sliding window.



Sobel methodology

Sobel Gradient Computation

- Sobel operator convolves a 3×3 kernel over each pixel to compute spatial gradients
- **Gx**: horizontal gradient kernel — detects vertical edges
- **Gy**: vertical gradient kernel — detects horizontal edges
- Gradient magnitude: $M = |Gx| + |Gy|$

3×3 Sliding Window

- A fully register-partitioned 3×3 window shifts one column per clock cycle
- Provides simultaneous access to all 9 neighboring pixels in a single cycle
- Enables **II=1 pipelined execution** — one output pixel per clock cycle

generated window

4	5	6
7	8	9
10	11	12

Sobel methodology

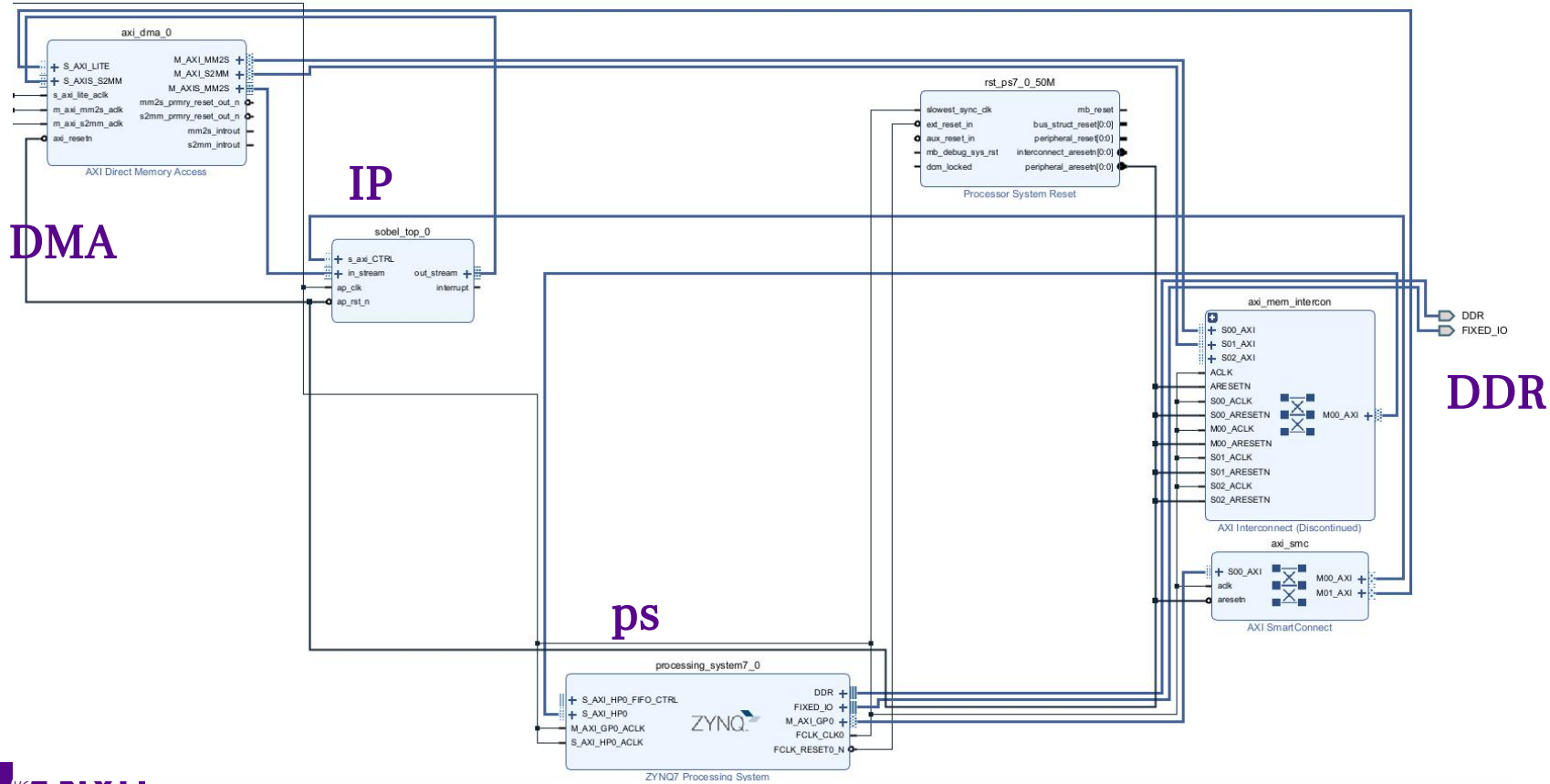
Streaming Input — No Frame Buffer Required

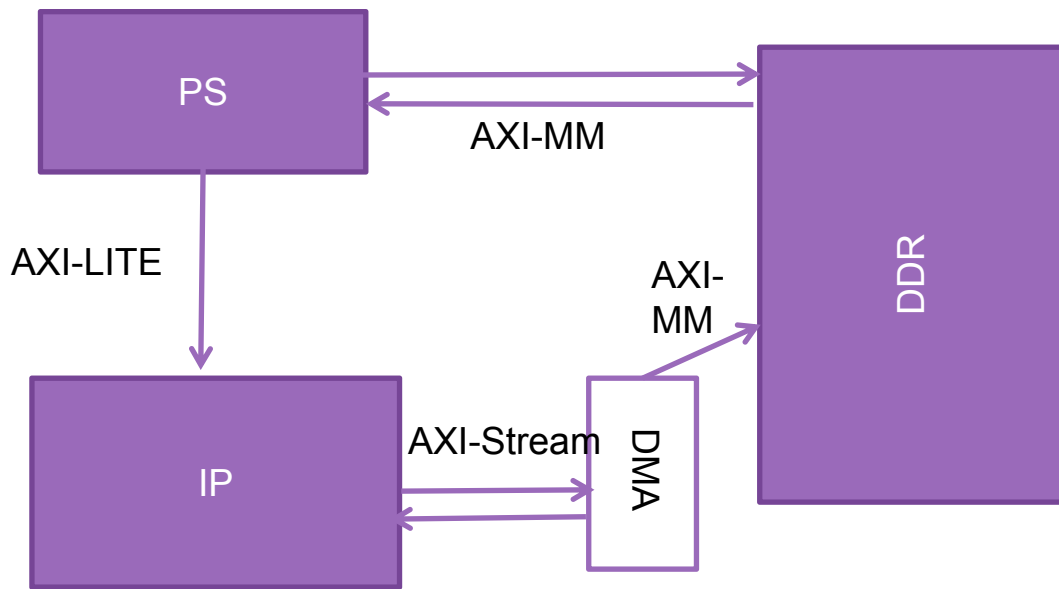
- Image is received as a **pixel-sequential AXI-Stream**, one byte per transaction
- Full-frame storage is avoided — only **two BRAM line buffers** retained at any time
- Each line buffer stores one complete row (up to 1920 pixels × 8-bit)
- At any cycle: row N-2 and row N-1 are buffered; row N arrives live from the stream

generated window

4	5	6
7	8	9
10	11	12

5	6	line buffer(previous column)
8	9	line buffer(previous column)
11	12	new pixel





Abstract graphic

AXI-Lite: used by the PS to configure the Sobel IP, including width, height, and control/status registers.

AXI-MM: used by the AXI DMA to read the input image from DDR and write the processed output image back to DDR.

AXI-Streaming: transfer the pixel data to IP FIFO/DMA FIFO, easy pipelining

```
#include "sobel.hpp" You, 2 weeks ago • main

void sobel_top(hls::stream<axis_t> &in_stream,
               hls::stream<axis_t> &out_stream,
               int width,
               int height) {
#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream

#pragma HLS INTERFACE s_axilite port=width bundle=CTRL
#pragma HLS INTERFACE s_axilite port=height bundle=CTRL
#pragma HLS INTERFACE s_axilite port=return bundle=CTRL

    sobel_core(in_stream, out_stream, width, height);
}
```


Storage definition

```
void sobel_core(hls::stream<axis_t> &in_stream,
               hls::stream<axis_t> &out_stream,
               int width,
               int height) {}
#pragma HLS INLINE off

if (width <= 0 || height <= 0) return;
if (width > MAX_WIDTH || height > MAX_HEIGHT) return;

ap_uint<8> linebuf0[MAX_WIDTH]; //bram
ap_uint<8> linebuf1[MAX_WIDTH];
    You, 2 weeks ago • main
#pragma HLS BIND_STORAGE variable=linebuf0 type=ram_2p impl=bram
#pragma HLS BIND_STORAGE variable=linebuf1 type=ram_2p impl=bram

ap_uint<8> window[3][3]; //partition into register
#pragma HLS ARRAY_PARTITION variable=window complete dim=0
```

2Bram: Dual port
support read and
write simultaneously

9 registers

3*3 window generate & linebuffer update

```
#include "sobel.hpp" You, 2 weeks ago • main

void window_generator(ap_uint<8> pix,
                     ap_uint<11> col,
                     ap_uint<8> linebuf0[MAX_WIDTH],
                     ap_uint<8> linebuf1[MAX_WIDTH],
                     ap_uint<8> window[3][3]) {
#pragma HLS INLINE

    ap_uint<8> old0 = linebuf0[col];
    ap_uint<8> old1 = linebuf1[col];

    window[0][0] = window[0][1];
    window[0][1] = window[0][2];
    window[1][0] = window[1][1];
    window[1][1] = window[1][2];
    window[2][0] = window[2][1];
    window[2][1] = window[2][2];

    window[0][2] = old0;
    window[1][2] = old1;
    window[2][2] = pix;

    linebuf0[col] = old1;
    linebuf1[col] = pix;
}
```

Pipeline and algorithm

```
row_loop:
    for (int row = 0; row < height; row++) {
col_loop:
    for (int col = 0; col < width; col++) {
#pragma HLS PIPELINE II=1

        axis_t in_pkt = in_stream.read();
        ap_uint<8> pix = in_pkt.data;

        ap_uint<11> col_u = col;
        window_generator(pix, col_u, linebuf0, linebuf1, window);

        ap_uint<8> out_pix = 0;

        if (row >= 2 && col >= 2) {
            ap_int<12> p00 = window[0][0];
            ap_int<12> p01 = window[0][1];
            ap_int<12> p02 = window[0][2];

            ap_int<12> p10 = window[1][0];
            ap_int<12> p12 = window[1][2];

            ap_int<12> p20 = window[2][0];
            ap_int<12> p21 = window[2][1];
            ap_int<12> p22 = window[2][2];

            ap_int<12> gx_left  = p00 + (p10 << 1) + p20;
            ap_int<12> gx_right = p02 + (p12 << 1) + p22;
            ap_int<12> gx      = gx_right - gx_left;

            ap_int<12> gy_top   = p00 + (p01 << 1) + p02;
            ap_int<12> gy_bot   = p20 + (p21 << 1) + p22;
            ap_int<12> gy      = gy_bot - gy_top;

            ap_uint<12> mag = abs12(gx) + abs12(gy);
            out_pix = clip_u8_12(mag);
        }
    }
}
```

- Instead of buffering the full frame, only two preceding rows are stored in BRAM, while the active 3×3 neighborhood is held in fully partitioned registers for single-cycle parallel access.
- Applying pipeline to the outer row loop introduces loop-carried dependencies on line buffer updates, preventing II=1; the directive is therefore applied to the inner column loop, where all dependencies resolve within a single iteration.

HLS Optimizations

#pragma HLS PIPELINE II=1

Applied to the inner column loop, enabling one pixel processed per clock cycle with full spatial reuse of line buffer data.

#pragma HLS ARRAY_PARTITION variable=window complete dim=0

Completely partitions the 3×3 window across all dimensions into dedicated registers, eliminating memory port contention and exposing all nine pixels to the datapath in a single cycle.

Bit-Width Optimization

Sobel gradients on 8-bit inputs require at most 12 bits. Explicit **ap_int<12>** specification avoids default 32-bit arithmetic, reducing LUT utilization and shortening the critical path.

Synthesis / Resource Results

Target clock: 8.00 ns / 125 MHz

Estimated clock: 5.799 ns / ~172 MHz

II: 1

BRAM_18K: 2

DSP: 1

FF: 1,197

LUT: 1,837

The target was 125 MHz. HLS estimated 5.799 ns, which is faster than the 8 ns target, so the design meets the HLS timing goal.

At II=1 and 125 MHz, the theoretical steady-state throughput is about 125 million pixels per second.

Resource usage is low because the design only keeps two line buffers and a 3×3 register window, instead of buffering the full frame.

Estimated Quality of Results

Timing Estimate



TARGET	WITH UNCERTAINTY	ESTIMATED
8.00 ns	5.84 ns	5.799 ns

Performance & Resource Estimates

⬆

⬇

☰

🏗

ⓘ

⚠

🔍

🔄

%

Search

🔍

🔼

☒ Modules

☒ Loops

☒ Hide empty columns

LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
2.951E7		3,688,338		no	function	2	1	1,197	1,839	0
2.951E7		3,688,334		no	function	2	1	1,002	1,572	0
1.538E4		1,921		loop pipeline stpst	function	0	0	25	62	0
1.536E4	2	1	1,920	yes	loop					
2.949E7		3,686,401		loop pipeline stpst	function	0	0	888	1,090	0
2.949E7	8	1	3,686,400	yes	loop					

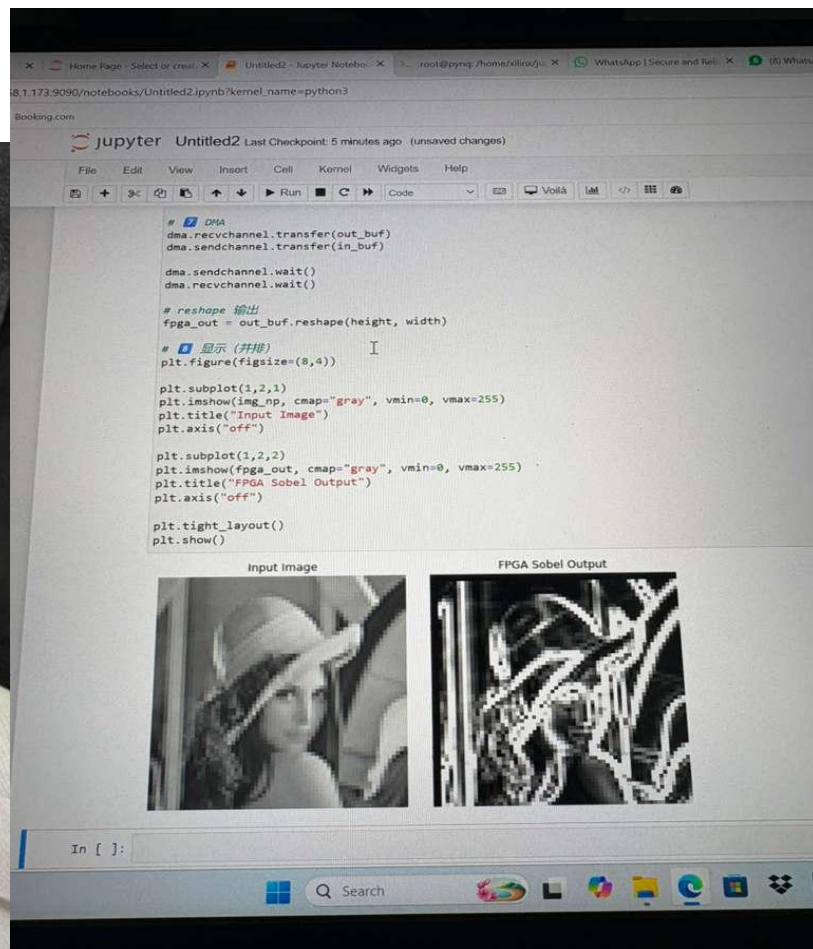
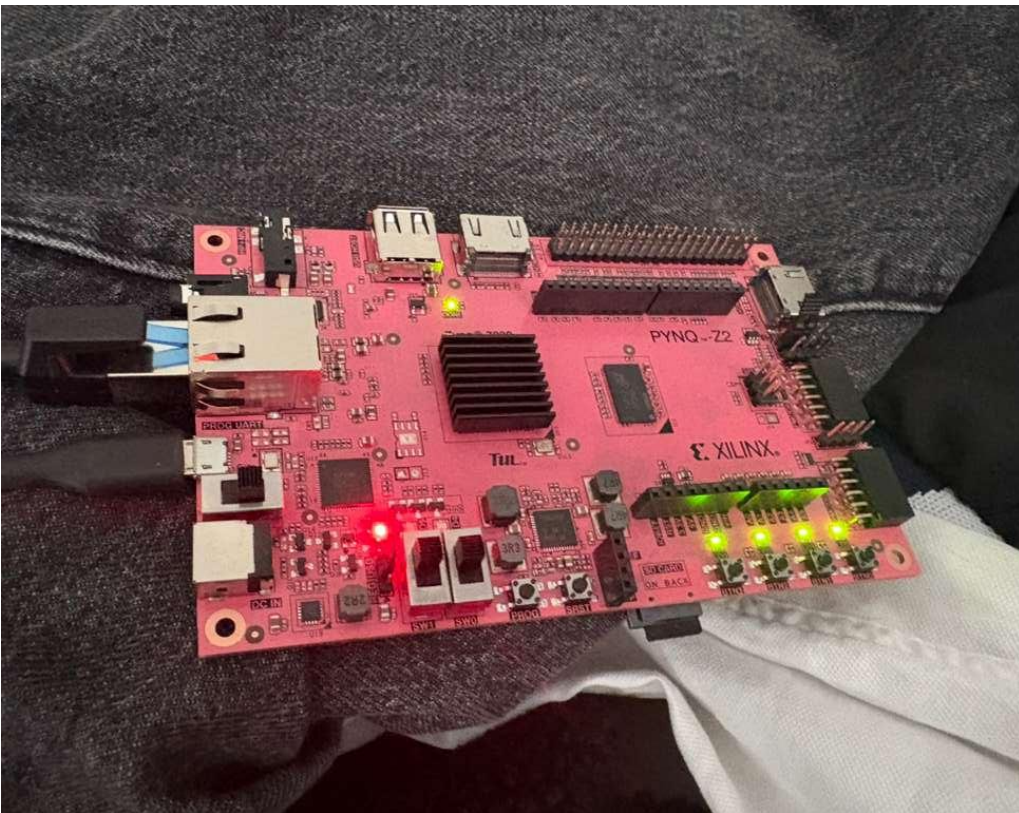

```
OUTPUT x PROBLEMS 3 x sobel::synthesis
Vitis Messages sobel::c-simulation sobel::synthesis x sobel::co-simulation sobel::package
INFO: [HLS 200-10] -- Generating RTL for module 'sobel_top'
INFO: [HLS 200-10] -----
WARNING: [RTGEN 206-101] Design contains AXI ports. Reset is fixed to synchronous and active low.
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_data_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_keep_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_strb_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_user_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_last_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_id_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/in_stream_V_dest_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_data_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_keep_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_strb_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_user_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_last_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_id_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/out_stream_V_dest_V' to 'axis' (register, both mode).
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/width' to 's_axilite & ap_none'.
INFO: [RTGEN 206-500] Setting interface mode on port 'sobel_top/height' to 's_axilite & ap_none'.
INFO: [RTGEN 206-500] Setting interface mode on function 'sobel_top' to 's_axilite & ap_ctrl_hs'.
INFO: [RTGEN 206-100] Bundling port 'width', 'height' and 'return' to AXI-Lite port CTRL.
INFO: [RTGEN 206-100] Finished creating RTL model for 'sobel_top'.
INFO: [HLS 200-111] Finished Creating RTL model: CPU user time: 0 seconds. CPU system time: 1 seconds. Elapsed time: 0.34 seconds; current alloc
INFO: [HLS 200-111] Finished Generating all RTL models: CPU user time: 0 seconds. CPU system time: 0 seconds. Elapsed time: 1.085 seconds; current
INFO: [HLS 200-2225] Wrote inferred directives to file C:/Users/57778/Desktop/sobel/sobel/sobel_top/hls/syn/inferred_directives.ini
INFO: [HLS 200-111] Finished Updating report files: CPU user time: 1 seconds. CPU system time: 2 seconds. Elapsed time: 3.007 seconds; current al
INFO: [VHDL 208-304] Generating VHDL RTL for sobel_top.
INFO: [VLOG 209-307] Generating Verilog RTL for sobel_top.
INFO: [HLS 200-789] **** Estimated Fmax: 172.44 Mhz
INFO: [HLS 200-112] Total CPU user time: 6 seconds. Total CPU system time: 12 seconds. Total elapsed time: 106.087 seconds; peak allocated memory
INFO: [v++ 60-791] Total elapsed time: 0h 1m 54s
Synthesis finished successfully. open report.
```

```
from hls_config.cfg(5)
p/sobel/sobel/sobel_top/vitis-comp.json
```

```
In file included from C:/AMDDesignTools/2025.2/Vitis/include/floating_point_v7_1_bitacc_cmodel.h:150:
C:/AMDDesignTools/2025.2/Vitis/include/gmp.h:58:9: warning: '___GMP_LIBGMP_DLL' macro redefined [-Wmacro-redefined]
#define ___GMP_LIBGMP_DLL 0
^
C:/AMDDesignTools/2025.2/Vitis/include/floating_point_v7_1_bitacc_cmodel.h:142:9: note: previous definition is here
#define ___GMP_LIBGMP_DLL 1
^
1 warning generated.
Input image size: 784 x 786
PASS: HLS output matches golden reference.
INFO [HLS SIM]: The maximum depth reached by any hls::stream() instance in the design is 616224
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] ***** CSIM finish *****
INFO: [HLS 200-112] Total CPU user time: 2 seconds. Total CPU system time: 6 seconds. Total elapsed time: 16.896 seconds; peak allocated memo
INFO: [vitis-run 60-791] Total elapsed time: 0h 0m 25s
C-simulation finished successfully
```



Final deploy





Thanks!

Lixuan Xu, Hao Ding, Yuhan Jiang
lx2349@nyu.edu, hd2891@nyu.edu, yj3494@nyu.edu